Agentic AI Framework for Context-Aware Semantic Understanding and Intelligent Assistance of Dynamic Web Forms

(Smart Form Auto-Filling System)

Submitted in partial fulfillment of the requirement for the award of Degree of Bachelor of Technology in Computer Science

Submitted To



SVKM's NMIMS,
Mukesh Patel School of Technology Management & Engineering,
Shirpur Campus (M.H.)

Submitted By:

Ashmit Patil Enrollment No – 70552300048
Sanket Mali Enrollment No – 70552300067
Pinak Salunke Enrollment No – 70552300049

Under The Supervision Of:

Dr. Nitin S. Choubey
(Head of Department, Computer Science)

DEPARTMENT OF COMPUTER SCIENCE
Mukesh Patel School of Technology Management & Engineering
ACADEMIC SESSION: 2026-27

CERTIFICATE

This is to certify that the project entitled **"Agentic AI Framework for Context-Aware Semantic Understanding and Intelligent Assistance of Dynamic Web Forms"** has been done by

"Ashmit Patil" (Roll No.: 246; Enrollment No.: 70552300048)

"Sanket Mali" (Roll No.: 265; Enrollment No.: 70552300067)

"Pinak Salunke" (Roll No.: 247; Enrollment No.: 70552300049)

under my guidance and supervision and has been submitted in partial fulfillment of the degree of "**Bachelor of Technology in Computer Science**" of SVKM's NMIMS (Deemed-to-be University), MPSTME Shirpur Campus (M.H.), India.

Project Mentor	**Examiner**
(Internal Guide)	
Dr. Nitin S. Choubey	

Date:

Place: Shirpur

H.O.D.
Dr. Nitin S. Choubey

DEPARTMENT OF COMPUTER SCIENCE
Mukesh Patel School of Technology Management & Engineering

CERTIFICATE OF APPROVAL

The project entitled "**Agentic AI Framework for Context-Aware Semantic Understanding and Intelligent Assistance of Dynamic Web Forms**", submitted by Ashmit Patil (70552300048), Sanket Mali (70552300067) and Pinak Salunke (70552300049) of B.Tech. (Computer Science), Academic Session 2026-27, MPSTME Shirpur Campus, SVKM's NMIMS, has been examined and is approved in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science.

The examiners record their approval below after the project presentation and viva-voce examination held on _____.

_____	_____
Internal Examiner	**External Examiner**
Name: _____	Name: _____
_____	_____
Project Guide	**Head of Department**
Dr. Nitin S. Choubey	Dr. Nitin S. Choubey

Date: _____

Place: Shirpur

DECLARATION

We, the undersigned students of B.Tech. (Computer Science), Academic Session 2026-27, MPSTME Shirpur Campus, SVKM's NMIMS, declare that the project report entitled "**Agentic AI Framework for Context-Aware Semantic Understanding and Intelligent Assistance of Dynamic Web Forms**" is a record of the work carried out by us under the guidance of Dr. Nitin S. Choubey, Head of Department, Computer Science. The software, diagrams, test results and screenshots in this report were produced from the implementation developed by the project team.

To the best of our knowledge, this report has not been submitted, in part or in full, to any other university or institution for the award of any degree or diploma. Information taken from published or unpublished sources has been acknowledged in the text and listed in the References.

_____ _____

Ashmit Patil **Sanket Mali**
Enrollment No. 70552300048 Enrollment No. 70552300067

_____

Pinak Salunke
Enrollment No. 70552300049

Date: _____

Place: Shirpur

ACKNOWLEDGEMENT

After the completion of this capstone project work, words are not enough to express our feelings about all those who helped us to reach our goal; above all, we are indebted to The Almighty for providing us this moment in life.

It is a great pleasure and a moment of immense satisfaction for us to express our profound gratitude to our project guide, **Dr. Nitin S. Choubey**, Head of Department, Computer Science, MPSTME, Shirpur, whose constant encouragement enabled us to work enthusiastically. Their motivation, patience and expert guidance during the progress of the project benefited us beyond expression, and their depth of knowledge of Computer Science showed us that sound theory is the basis of efficient working software. We are also grateful to them, as Head of Department, for providing the infrastructure and support needed to complete the project successfully.

We extend our deepest gratitude to **Dr. Venkatadri Marriboyina**, Associate Dean, SVKM's NMIMS, Shirpur Campus, for providing all the necessary facilities and an encouraging environment to bring out the best of our endeavours.

We sincerely thank all members of the staff of the Department of Computer Science, who imparted technical knowledge to us during the various stages of B.Tech. Computer Science, and the reviewers whose comments improved this report. We also thank our friends, who contributed directly or indirectly to this work, and the open-source communities whose tools made it possible.

Ashmit Patil
Sanket Mali
Pinak Salunke

ABSTRACT

People fill the same personal details into many web forms: admission portals, scholarship and examination applications, bank account forms and government services. The information already exists in documents they hold, such as identity cards, mark sheets and certificates, yet browser autofill can only repeat what was typed before. It cannot read documents, cannot show where a value came from, and does not warn when two documents disagree.

This report presents the Smart Form Auto-Filling System, a privacy-first assistant that builds a verified profile from a user's own documents and uses it to suggest values for any web form. The system has a FastAPI backend, a React dashboard and a Chrome extension. Uploaded PDF, Word, text and image files are read (with optical character recognition for scans), classified into seventeen Indian document types, and turned into facts from a vocabulary of 220 concepts. Every fact keeps its source document, page and evidence text. Facts are stored in MongoDB and the document text is indexed in ChromaDB for semantic search. On a web form, a LangGraph pipeline maps each field to a concept, proposes a value from the profile or from cited document text, and decides whether the value is ready or must be reviewed. The user approves every value before the extension fills it, and the form is never submitted automatically. A question-answering page answers questions about the documents with cited sources.

On 22 October 2026 all 446 automated tests that were run passed (301 backend, 3 vector-database, 30 dashboard and 112 extension tests), and the 7 browser end-to-end tests passed in the recorded run of 15 October 2026. On a synthetic set of 15 documents and 15 forms, 97.0% of expected fields were extracted correctly, all 28 test questions were answered correctly with sources, and 112 form fields were filled correctly, 9 incorrectly and 84 were held for the user's review, with no form submitted.

Keywords: form autofill, document information extraction, optical character recognition, retrieval-augmented generation, LangGraph, MongoDB, Chrome extension, privacy.

TABLE OF CONTENTS

Chapter 1 INTRODUCTION 1
1.1 BACKGROUND OF THE PROJECT TOPIC 1
1.2 MOTIVATION AND SCOPE OF THE PROJECT 2
1.3 PROBLEM STATEMENT 2
1.4 AIM AND OBJECTIVES 3
1.5 SALIENT CONTRIBUTIONS 3
1.6 ORGANISATION OF THE REPORT 4
CHAPTER SUMMARY 4
Chapter 2 LITERATURE SURVEY 5
2.1 INTRODUCTION TO THE OVERALL TOPIC 5
2.1.1 Basic Concepts Used in This Project 5
2.2 EXHAUSTIVE LITERATURE SURVEY 6
2.2.1 Web Agents and Benchmarks 6
2.2.2 Agents, Knowledge and Memory 7
2.2.3 Form Understanding, Form Filling and Form Security 7
2.2.4 Supporting Techniques and Security 8
2.3 COMPARATIVE ANALYSIS 9
2.4 RESEARCH GAP 9
CHAPTER SUMMARY 9
Chapter 3 PROBLEM DEFINITION & PROPOSED SOLUTION 10
3.1 PROBLEM DEFINITION 10
3.2 EXISTING SYSTEM AND ITS LIMITATIONS 11
3.3 PROPOSED SOLUTION 12
3.3.1 Workflow Summary 12
3.3.2 Advantages of the Proposed Solution 13
3.4 REQUIREMENTS 13
3.5 TECHNOLOGY STACK AND JUSTIFICATION 14
3.6 ASSUMPTIONS AND CONSTRAINTS 14
CHAPTER SUMMARY 15
Chapter 4 DESIGN 16
4.1 SYSTEM ARCHITECTURE 16
4.2 DATA FLOW 19
4.3 DATABASE DESIGN 19
4.4 FORM-FILLING PIPELINE 21
4.5 DECISION RULES 22
4.6 SECURITY DESIGN 23
CHAPTER SUMMARY 23
Chapter 5 RESULT ANALYSIS: SNAPSHOTS 24
5.1 DASHBOARD AND DOCUMENT PROCESSING 24
5.2 PROFILE AND CONFLICTS 26
5.3 ASK DOCUMENTS 28
5.4 BROWSER EXTENSION 29

5.5 MEASURED RESULTS 30
5.6 ANALYSIS 31
CHAPTER SUMMARY 32
Chapter 6 TESTING 33
6.1 TESTING STRATEGY 33
6.2 TEST ENVIRONMENT 33
6.3 TEST INVENTORY 34
6.4 TEST CASES 34
6.5 TEST RESULTS 35
6.6 DISCUSSION 36
CHAPTER SUMMARY 36
Chapter 7 CONCLUSION & FUTURE WORK 37
7.1 CONCLUSION 37
7.1.1 Achievement of Objectives 37
7.1.2 Principal Findings 38
7.2 LIMITATIONS AND FUTURE WORK 38
CHAPTER SUMMARY 38
References 39
Appendix I: Source Code Excerpts 43
I.1 Construction of the form-filling graph 43
I.2 Prompt-injection patterns 44
I.3 Writing a value into a web page 45
Appendix II: List of Components 46
II.1 Installing and running the system 46
II.2 Main API endpoints 48
Appendix III: List of Papers Presented and Published 49

LIST OF FIGURES

Figure 3.1: Two-phase workflow of the proposed solution 12
Figure 4.1: End-to-end architecture and data flow, from document upload to reviewed form filling 17
Figure 4.2: Data-flow diagram, level 1 19
Figure 4.3: Entity–relationship diagram of the MongoDB collections and the ChromaDB chunk index 20
Figure 4.4: Flowchart of the decide step of the form-filling pipeline 22
Figure 5.1: Dashboard overview of the demonstration account 24
Figure 5.2: Facts extracted from the Class 12 mark sheet, with evidence 25
Figure 5.3: Profile page with the supporting documents of each value 26
Figure 5.4: Conflict between two dates found in one certificate 27
Figure 5.5: Ask Documents: answer with the cited source 28
Figure 5.6: Extension popup: review before filling (left) and verified result (right) 29
Figure 5.7: Filled subject-wise marks on the synthetic Class 12 form 30

LIST OF TABLES

Table 1.1: Application domains of the system....2
Table 2.1: Summary of the surveyed literature....8
Table 2.2: Comparison of existing approaches with the proposed system....9
Table 3.1: Existing practice compared with the proposed system....11
Table 3.2: Functional requirements....13
Table 3.3: Technology stack and justification....14
Table 5.1: Extraction, retrieval and question-answering results (15 synthetic documents)....30
Table 5.2: Browser autofill results on 15 synthetic forms....31
Table 6.1: Automated test inventory....34
Table 6.2: Representative test cases....35
Table 6.3: Test results: total, passed and failed test cases....35
Table 7.1: Achievement of the objectives....37
Table 7.2: Limitations and proposed future work....38
Table II.1: Software components and versions....46
Table II.2: Commands that run the automated test suites....47
Table II.3: Main API endpoints used in the workflow....48

LIST OF ABBREVIATIONS

Abbreviation	Full form
AI	Artificial Intelligence
API	Application Programming Interface
CAPTCHA	Test that tells computers and humans apart
CI	Continuous Integration
CPU	Central Processing Unit
CSS	Cascading Style Sheets
DFD	Data-Flow Diagram
DOCX	Microsoft Word document format
ER	Entity–Relationship
FR	Functional Requirement
HTTP	Hypertext Transfer Protocol
ICLR / ICML	Int. Conf. on Learning Representations / on Machine Learning
JSON	JavaScript Object Notation
JWT	JSON Web Token
KYC	Know Your Customer
LLM	Large Language Model
MV3	Manifest Version 3 (Chrome extension platform)
OCR	Optical Character Recognition
ONNX	Open Neural Network Exchange
OTP	One-Time Password
OWASP	Open Worldwide Application Security Project
PDF	Portable Document Format
PIN	Postal Index Number
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
TC	Test Case
UI	User Interface
UML	Unified Modelling Language

Chapter 1
INTRODUCTION

This chapter introduces the Smart Form Auto-Filling System. It explains why filling web forms from personal documents is a problem worth solving, states the scope, problem, aim and objectives of the project, lists its main contributions, and describes how the rest of the report is organised.

1.1 BACKGROUND OF THE PROJECT TOPIC

Almost every service that a student or a working adult uses today begins with an online form. Admission and scholarship portals ask for marks, board names and addresses; examination portals ask for the same name, date of birth and category details again; banks ask for identity numbers and income. Most of these facts already exist in documents the person holds: an Aadhaar or PAN card, Class 10 and Class 12 mark sheets, a birth certificate, an income or domicile certificate, or a bank passbook.

Web browsers try to reduce this effort with **autofill**, a feature that remembers values typed earlier and offers them again when a field looks similar. Autofill helps with an e-mail address or a postal address, but it has three limits. First, it can only repeat values the user once typed, so it cannot use the facts printed in documents. Second, it matches fields mainly by their technical names, so a label such as “Year of passing (Class XII)” or “Father’s name” is easily confused with a similar field. Third, it keeps no record of where a value came from. Autofill can also leak data: Lin, Ilia and Polakis showed that web pages can hide fields that browser autofill fills silently [1].

Recent progress in artificial intelligence (AI) makes a better approach possible. A **large language model (LLM)** is an AI model trained on very large amounts of text; it can read an unstructured document and return structured information. **Retrieval-augmented generation (RAG)** lets such a model answer from passages of the user’s own documents instead of from general knowledge [2]. Research on web agents [3], [4], [5] has shown that models can operate real web pages, but those agents act on their own. Form filling with personal data needs the opposite design: a careful

assistant that proposes values from verified documents, explains each proposal and leaves every decision to the user. This project builds such an assistant.

1.2 MOTIVATION AND SCOPE OF THE PROJECT

Re-typing the same details is slow and error-prone. A wrong digit in an identity number, a date written in the wrong format, or marks entered from the wrong examination can lead to a rejected application. When two documents give different values, for example two spellings of a name, a simple autofill tool picks one without telling the user. The project therefore has three aims at once: reduce typing, keep every value traceable to evidence, and keep the user in control. Table 1.1 lists the kinds of portal on which the same facts recur.

Table 1.1: Application domains of the system

Domain	Typical form	Facts that recur
Education	Admission, scholarship and examination applications	Name, date of birth, board, seat number, subject marks, percentage
Government services	Certificates, voter and passport services	Name, parents' names, address, identity numbers, category
Banking	Account opening and KYC (know-your-customer) update	Identity numbers, address, income, account details
Employment	Recruitment portals	Education, experience, skills, contact details

In scope: upload and processing of PDF, Word (DOCX), plain-text and image files; optical character recognition (OCR) for scanned images; seventeen common Indian document types; a profile with sources and conflict resolution; a Chrome browser extension that reads the fields of any web form, suggests values and fills the approved ones; and a question-answering page over the user's documents.

Out of scope: automatic submission of forms; filling passwords, payment-card details, one-time passwords (OTPs) and CAPTCHAs; browsers other than Chrome; mobile applications; and reading text in Indian scripts, because the OCR models used read English text only.

1.3 PROBLEM STATEMENT

Existing autofill tools cannot build a profile from a person's documents, cannot show evidence for the values they insert, handle disagreeing sources poorly, and in some designs fill hidden or sensitive fields. The problem addressed by this project is to design and implement a system that (a) extracts structured facts with evidence from

personal documents, (b) matches the fields of any web form to those facts by meaning rather than by exact field names, (c) detects and shows disagreements between documents, and (d) fills values only after the user approves them, without reading what the page already contains, touching sensitive fields or submitting the form.

1.4 AIM AND OBJECTIVES

The aim of the project is to build a privacy-first assistant that fills web forms from the user's own documents, with every value traceable and every decision taken by the user. The objectives are:

1. **Document understanding with evidence:** read PDF, Word, text and scanned image documents, recognise the type of document, and extract personal facts into a fixed vocabulary, recording for every fact its source document, page, evidence text and confidence.
2. **A verified profile:** combine the facts from all documents into one profile, detect when documents disagree, and let the user resolve each disagreement.
3. **Semantic form filling:** recognise what each field of an unseen web form asks for, propose a value from the profile or from cited document text, and fill only the values the user approves.
4. **Grounded question answering:** answer the user's questions about their own documents and show the document and page behind every answer.
5. **Safety and verification:** protect personal data (no automatic submission, sensitive fields skipped, defences against manipulated text, per-user isolation) and verify the system with automated tests and a measured evaluation.

1.5 SALIENT CONTRIBUTIONS

- **One shared vocabulary:** an ontology of 220 concepts in 32 domains, with 645 alternative phrasings (aliases), is used by extraction, field matching and filling, so that all parts of the system mean the same thing by "Class 12 percentage" or "father's name".
- **Evidence for every value:** each fact stores its document, page, the exact text it came from and a confidence score, and disagreements between documents become visible conflicts instead of silent choices.

- **Rules before models:** fixed rules handle the common cases; the language model is called only for fields the rules cannot decide, and any value it proposes from a document must appear word for word in the cited text.
- **User-approved filling:** the browser extension never reads existing values, skips sensitive fields, shows a reason and source for every suggestion, fills only ticked fields, checks that the page kept them, and never submits the form.
- **Measured results:** 453 automated tests and an evaluation on 15 synthetic documents and 15 forms report what works and where the system still makes mistakes.

1.6 ORGANISATION OF THE REPORT

Chapter 2 explains the basic concepts used in the project and surveys published research on web agents, document understanding, form filling and security, ending with the research gap. Chapter 3 defines the problem, compares existing practice with the proposed solution and lists the requirements and technologies. Chapter 4 presents the design: architecture, data flow, database, the form-filling pipeline and its decision rules. Chapter 5 shows snapshots of the working system and analyses the measured results. Chapter 6 describes testing and the test results. Chapter 7 gives the conclusion and future work, followed by the References and Appendices.

CHAPTER SUMMARY

Online forms ask again and again for facts that already exist in a person’s documents, but browser autofill cannot use those documents, cannot show evidence and cannot handle disagreements. This project builds an assistant that extracts facts with evidence, matches form fields by meaning and fills only what the user approves. The scope, problem, five objectives and contributions were stated.

Chapter 2
LITERATURE SURVEY

This chapter first explains, in simple terms, the concepts on which the project is built. It then reviews published research grouped by theme, compares the main approaches with the proposed system, and ends with the gap that the project addresses.

2.1 INTRODUCTION TO THE OVERALL TOPIC

Automatic form filling from documents combines four research areas. **Document understanding** turns scanned or digital documents into structured facts; recent reviews show a move from layout rules to transformer models [6], and practical OCR systems such as PP-OCR read text from images on ordinary computers [7]. **Retrieval-augmented generation** grounds a language model's answer in retrieved passages [2], using sentence embeddings such as Sentence-BERT [8] and compact models such as MiniLM [9]. **Web agents** use language or vision-language models to act on web pages [3], [4], [5], [10], and surveys organise them into perception, planning and action [11], [12]. **Security research** shows how autofill and LLM-based applications can be abused [1], [13], [14].

2.1.1 Basic Concepts Used in This Project

Database and MongoDB. A database is an organised store of data that a program can save, search and update reliably. Traditional relational databases keep data in tables with fixed columns. MongoDB is a *document database*: it stores each record as a flexible document of field–value pairs, similar to JSON (JavaScript Object Notation), and groups similar documents into collections [15]. Records of different shapes can therefore sit side by side, which suits facts as varied as a date of birth and a table of subject marks. In this project MongoDB stores user accounts, uploaded-document records, extracted facts with their evidence, and form-filling sessions.

Embeddings and vector databases. An embedding is a list of numbers that represents the meaning of a piece of text; texts with similar meaning have nearby embeddings [8]. A vector database stores these lists and quickly finds the ones closest

to a query. The project uses ChromaDB [16] to store embeddings of short passages (chunks) of each document, so that a question such as "What was my Class 12 percentage?" finds the right passage even if the words differ.

Large language models and RAG. An LLM predicts text and can therefore answer questions or turn text into structured data, but it can also invent facts. RAG reduces this risk by giving the model only retrieved passages and asking it to answer from them [2]. The project goes one step further: a value proposed from a document must appear word for word in the passage the model cites.

Other building blocks. *OCR* converts an image of text into characters [7]. An *ontology* is a controlled list of concepts, such as *personal.date_of_birth*, with their alternative names and data types. A *REST API* (Representational State Transfer Application Programming Interface) lets the dashboard and the extension talk to the server over HTTP; the project builds it with FastAPI [17]. *LangGraph* runs a program as a graph of steps that share a state [18]. A *Chrome extension* is a small program added to the browser; Manifest V3 is the current, permission-controlled extension platform [19]. Finally, *prompt injection* is text hidden in data that tries to give instructions to an LLM [13].

2.2 EXHAUSTIVE LITERATURE SURVEY

2.2.1 Web Agents and Benchmarks

WebArena [3] provides a self-hosted environment of realistic websites and long tasks, and showed that LLM agents were far below human performance. Mind2Web [4] collected more than 2,000 tasks from 137 real websites and proposed a two-stage method in which a small model first filters page elements before a larger model chooses an action. SeeAct [10] studied GPT-4V as a web agent and found that *grounding*, linking a textual plan to the correct element on the page, is the main difficulty. WebVoyager [5] built an end-to-end multimodal agent for fifteen live websites. CogAgent [20] is an 18-billion-parameter vision-language model that reads high-resolution screenshots, and SeeClick [21] pre-trains models to locate interface elements from screenshots. Ferret-UI [22] extends this to mobile screens. WorkArena [23] measures agents on enterprise knowledge-work tasks and concludes that they are promising but not yet reliable, while LCoW [24] learns to simplify complex pages so that an agent decides better.

Relevance and limits. These works show that models can act on web pages and that finding the correct element is hard. The project avoids the grounding problem for form fields by reading the page structure directly: each field has a stable identifier, and filling targets that identifier. Unlike an autonomous agent, the extension performs no action the user has not approved.

2.2.2 Agents, Knowledge and Memory

Wang *et al.* [12] survey LLM-based agents and describe them in terms of profile, memory, planning and action modules, and Nguyen *et al.* [11] survey agents that operate graphical interfaces. Zhang *et al.* [25] organise agent memory by source, form and operation; in these terms, the project's profile is a persistent memory in which every item is linked to evidence. Pan *et al.* [26] argue that explicit, structured knowledge complements the knowledge hidden inside a model, and HippoRAG [27] combines a model, a knowledge graph and graph ranking to improve retrieval across several facts. The project's ontology plays the role of a small explicit knowledge structure, while its questions are mostly direct look-ups for which single-step retrieval is sufficient.

2.2.3 Form Understanding, Form Filling and Form Security

Abdallah *et al.* [6] review transformer models for understanding scanned forms, including key-information extraction and benchmark datasets; this is the reading side of the problem. FormLM [28] helps designers *create* online forms by modelling their structure and text. LAFF [29] suggests values for categorical fields in data-entry forms by learning dependencies between fields from earlier entries. FormNexus [30] infers the meaning and constraints of web-form fields from their text, structure and position, using an LLM, in order to generate tests. Lin *et al.* [1] show that browser autofill can be exploited through hidden fields and its preview feature in Chromium-based browsers.

Relevance and limits. These studies show that the meaning of a field can be inferred from its label and surroundings, which the project does with ontology aliases and nearby text. None of them builds a personal profile from documents or asks the user to review each value, and the autofill study directly motivates three rules of the project: sensitive and hidden fields are skipped, existing values are never read, and nothing is filled before approval.

2.2.4 Supporting Techniques and Security

Lewis *et al.* [2] introduced RAG by pairing a text generator with a searchable index of passages. Sentence-BERT [8] produces sentence embeddings that can be compared quickly by cosine similarity, and MiniLM [9] compresses large transformer models; the project's embedding model, all-MiniLM-L6-v2, belongs to this family. PP-OCR [7] describes the lightweight text detection and recognition models that the project runs through RapidOCR [31]. Greshake *et al.* [13] demonstrated *indirect* prompt injection, in which instructions placed in web pages or documents take control of an LLM-based application, and the OWASP guidance lists prompt injection as the first risk for such applications [14]. These works justify the project's retrieval design and its treatment of every page and document as untrusted input. Table 2.1 summarises the surveyed papers.

Table 2.1: Summary of the surveyed literature

Ref.	Work (venue, year)	Main idea	Limitation for this project
[3]	WebArena (ICLR 2024)	Realistic self-hosted web tasks	Task automation; no personal data or approval
[4]	Mind2Web (NeurIPS 2023)	2,000+ tasks on 137 websites	Agent actions, not form values
[10]	SeeAct (ICML 2024)	GPT-4V web agent; grounding is hard	Autonomous; visual grounding
[5]	WebVoyager (ACL 2024)	Multimodal agent on live sites	Autonomous; no evidence trail
[20]	CogAgent (CVPR 2024)	Screenshot-based 18B model	Heavy model; screenshots only
[21]	SeeClick (ACL 2024)	GUI grounding pre-training	Locates elements, not meaning
[23]	WorkArena (ICML 2024)	Enterprise task benchmark	Benchmark; agents not reliable
[27]	HippoRAG (NeurIPS 2024)	Graph-based memory retrieval	Multi-step retrieval not needed
[6]	Form understanding review (2024)	Models for scanned forms	Reads forms; does not fill them
[1]	Fill in the Blanks (CCS 2020)	Privacy threats of autofill	Analyses threats; no solution
[28]	FormLM (EMNLP 2022)	Helps create online forms	Form design, not filling
[29]	LAFF (ACM TOSEM 2023)	Predicts categorical values	Uses past entries, not documents
[30]	FormNexus (ISSTA 2024)	LLM infers field constraints	Testing, not user assistance
[13]	Indirect injection (AISec 2023)	Attacks via retrieved content	Shows the risk; no assistant

2.3 COMPARATIVE ANALYSIS

Table 2.2 compares the main families of approach on the properties that matter for this project, based on what the cited papers report.

Table 2.2: Comparison of existing approaches with the proposed system

Property	Browser autofill	Web agents [3], [5], [10]	Form research [28], [29], [30]	Proposed system
Reads the user's documents	No	No	No	Yes, including scans
Keeps evidence for each value	No	No	No	Document, page, text
Handles disagreeing sources	No	No	No	Shows conflict; user decides
Matches fields by meaning	Partly	Yes (model)	Yes	Ontology rules, then LLM
User approves before action	Partly	Usually not	Not applicable	Always, per field
Skips sensitive or hidden fields	Known weakness [1]	Not reported	Not applicable	Yes, twice

2.4 RESEARCH GAP

Web agents act on pages but are designed for autonomy, not for reviewing personal data. Form-related research infers field meaning for form design, value prediction or testing, but not from a person's own documents. Document-understanding research reads documents but does not connect them to forms, and security research documents the risks without offering a safe assistant. **Among the works reviewed, none combines a document-derived profile with evidence, conflict handling between documents, field matching by meaning, protection against prompt injection and mandatory user review in one deployable system.** This project addresses that combination.

CHAPTER SUMMARY

The basic concepts used in the project were explained, including databases and MongoDB, embeddings, RAG and OCR. Twenty-four published works, most of them peer-reviewed journal and conference papers, were reviewed: they show that models can understand and act on web pages, that grounding is hard, that autofill has privacy risks and that retrieval improves factual answers. No reviewed work offers evidence-based, conflict-aware and user-approved form filling from personal documents, which defines the problem of Chapter 3.

Chapter 3
PROBLEM DEFINITION & PROPOSED SOLUTION

This chapter turns the gap found in Chapter 2 into a precise problem, compares current practice with the proposed solution, describes the solution's workflow, and lists the requirements and the technologies chosen to meet them.

3.1 PROBLEM DEFINITION

A user uploads a set of personal documents. The system must derive from them a set of **facts**, where each fact records a concept from the ontology (for example *Class 12 percentage*), its value, the source document and page, the evidence text and a confidence score. Later, for any web form the user opens, the system must produce for each field a **recommendation**: a status (ready, needs review, missing or blocked), an optional value, a reason and a source. A solution is acceptable only if it meets the following constraints:

1. **Evidence:** a value is proposed only if it is a stored fact or appears word for word in a document passage that the system can cite.
2. **Conflict transparency:** if documents give different values for a single-valued concept, nothing is filled automatically; the candidates are shown.
3. **Sensitivity:** passwords, payment details, OTPs and CAPTCHAs are never filled, and identity or financial values always need confirmation.
4. **Consent:** the page changes only after the user approves each field, and the system never submits the form.
5. **Isolation and untrusted input:** no user can see another user's data, and text from web pages and documents is treated as data, never as instructions.

3.2 EXISTING SYSTEM AND ITS LIMITATIONS

Today a person types every value by hand, copies values from documents, or relies on browser autofill. Table 3.1 compares this practice with the proposed system on the points that matter for the problem definition.

Table 3.1: Existing practice compared with the proposed system

Aspect	**Existing practice (manual typing or browser autofill)**	**Proposed system**
Source of values	Typed by the user or remembered from earlier forms	Facts extracted from the user's own documents
Matching fields	Technical field names and simple rules	Ontology aliases and nearby text, then an LLM for unclear fields
Evidence	None	Document, page, evidence text and confidence
Disagreeing values	One value chosen without notice	Conflict shown; the user chooses
Sensitive and hidden fields	May be filled silently [1]	Skipped by the extension and blocked again by the server
After filling	No check	Each value read back from the page and reported
Submission	By the user or the page	Never by the system

3.3 PROPOSED SOLUTION

The solution works in two phases, shown in Figure 3.1. Phase A runs once for each uploaded document and builds the profile. Phase B runs every time the user opens a form and turns the profile into reviewed suggestions.

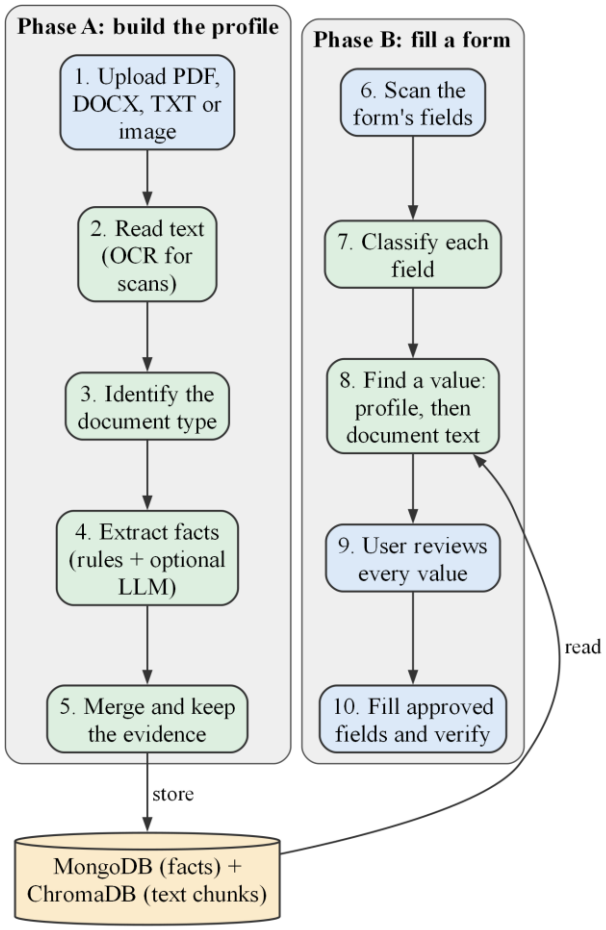


Figure 3.1: Two-phase workflow of the proposed solution

3.3.1 Workflow Summary

The workflow has two parts. First, the user uploads each document once. The system reads the text, identifies the type of document, extracts the facts, merges them with the facts already stored, and indexes the text so that it can be searched later. Because this processing can take several seconds, it runs in the background, and the dashboard shows its progress.

Second, whenever the user opens a form, the extension sends a description of the form's fields to the server. The server classifies each field, looks for a value in the profile and then in the document text, and decides whether the value is ready to use or needs the user's review. The extension shows these suggestions, fills only the fields

the user ticks, and checks that the page kept each value. The same pipeline serves every client, so the extension and the dashboard always give the same result for the same form.

3.3.2 Advantages of the Proposed Solution

- **Traceable:** every suggestion states where it came from and how certain the system is.
- **Fewer errors:** values are formatted to what each field expects, such as a date pattern or one option of a drop-down list.
- **Safe by design:** sensitive and hidden fields are skipped in two places, and the form is never submitted by the system.
- **Low cost:** the LLM is called only when the rules cannot decide, and the system still works, with fewer features, without any LLM.

3.4 REQUIREMENTS

Table 3.2 lists the functional requirements and the part of the system that implements each.

Table 3.2: Functional requirements

ID	Functional requirement	Implemented in
FR1	Register, sign in, sign out and delete the account	Authentication service
FR2	Upload PDF, DOCX, TXT, PNG or JPG files with type, size and duplicate checks	Document service
FR3	Process documents in the background and show stage and progress	Celery worker and pipeline
FR4	Read text, using OCR for scanned images	Text extractor (RapidOCR)
FR5	Identify the document type (17 Indian types)	Document registry
FR6	Extract facts with source, page, evidence and confidence	Rule-based and LLM extractors
FR7	View, add, edit and confirm profile values; resolve conflicts	Fact service and dashboard
FR8	Answer questions about the documents with cited sources	Ask Documents service
FR9	Detect form fields, skipping sensitive and hidden ones	Extension content script
FR10	Map fields to concepts and recommend values with reasons	Form-fill LangGraph pipeline
FR11	Record the user’s decisions; fill approved values and verify them	Extension popup and content script
FR12	Delete all data, with or without the account; report service health	User-data and health services

The main non-functional requirements are: **security** (hashed passwords, short-lived tokens, per-user isolation); **privacy** (the extension reads field descriptions, never field

values); **reliability** (failures are reported, never hidden behind invented values); **responsiveness** (form analysis returns at once and the popup checks progress every second); **maintainability** (automated tests and a continuous-integration workflow); and **portability** (the whole system starts with one Docker Compose command).

3.5 TECHNOLOGY STACK AND JUSTIFICATION

Table 3.3 lists the technology used in each layer and the reason for choosing it. MongoDB, defined in Section 2.1.1, was chosen because each fact has a different shape and every query is made for one user.

Table 3.3: Technology stack and justification

Layer	Technology	Reason for the choice
Server (API)	Python 3.11, FastAPI, Pydantic [17]	Fast asynchronous web framework with typed, validated requests
Fact store	MongoDB 8 [15]	Flexible documents; efficient per-user queries
Search index	ChromaDB [16] with all-MiniLM-L6-v2 [8], [9]	Self-hosted vector search; small embedding model that runs on a CPU
OCR	RapidOCR on ONNX Runtime [31] (PP-OCR models [7])	Accurate English OCR with low memory use
Form pipeline	LangGraph [18]	Explicit graph of steps with conditions
Language model	Google Gemini; optional local model through Ollama	Best measured accuracy; local fallback keeps working offline
Background jobs	Celery with Redis	Long processing does not block the user
Security	bcrypt [32], JSON Web Tokens [33]	Salted password hashing; short-lived signed tokens
Dashboard	React 19, TypeScript, Vite [34]	Component-based, type-checked user interface
Extension	Chrome Manifest V3, React 18 [19]	Current extension platform with on-demand page access
Deployment	Docker Compose, Nginx [35]	One command starts every service
Testing	pytest, Vitest, Playwright [36]	Unit, integration and real-browser tests

3.6 ASSUMPTIONS AND CONSTRAINTS

- Documents are in English (Indian bilingual documents are read from their English half); handwriting and Indian scripts are not read.
- A configured LLM improves classification of unusual fields and the wording of answers; without one, rule-based extraction, matching and filling still work.
- The system runs on one computer: development and all tests used a laptop with an Intel Core i5-13450HX processor and 16 GB of memory (15.7 GB usable), with no graphics processor needed.

- The user is responsible for the final content of every form; the system assists but never submits.

CHAPTER SUMMARY

The problem was defined with five acceptance constraints. Current practice was compared with the proposed two-phase solution, whose workflow and advantages were explained. Twelve functional requirements, the non-functional requirements and the technology stack with its justification were listed, together with the main assumptions.

Chapter 4
DESIGN

This chapter presents the design of the system as implemented: the overall architecture and the flow of data, the database design, the form-filling pipeline and its decision rules, and the security design. Each diagram is explained beside it.

4.1 SYSTEM ARCHITECTURE

The system follows a three-tier structure. The **presentation tier** consists of the two clients that the user sees: the web dashboard and the Chrome extension. The **application tier** is the FastAPI backend together with a background worker, which together carry out every operation on the user's data. The **data tier** holds the uploaded files, the facts and the searchable document chunks. External services, the LLM provider and the OCR models, are used only by the application tier and never receive more data than a task needs. The clients never access a database directly: every request passes through the backend, which checks who is asking before any data is read or written.

Figure 4.1 shows the complete architecture, from the clients through document ingestion and storage to the services that use the stored data. The numbered regions are described below the figure.

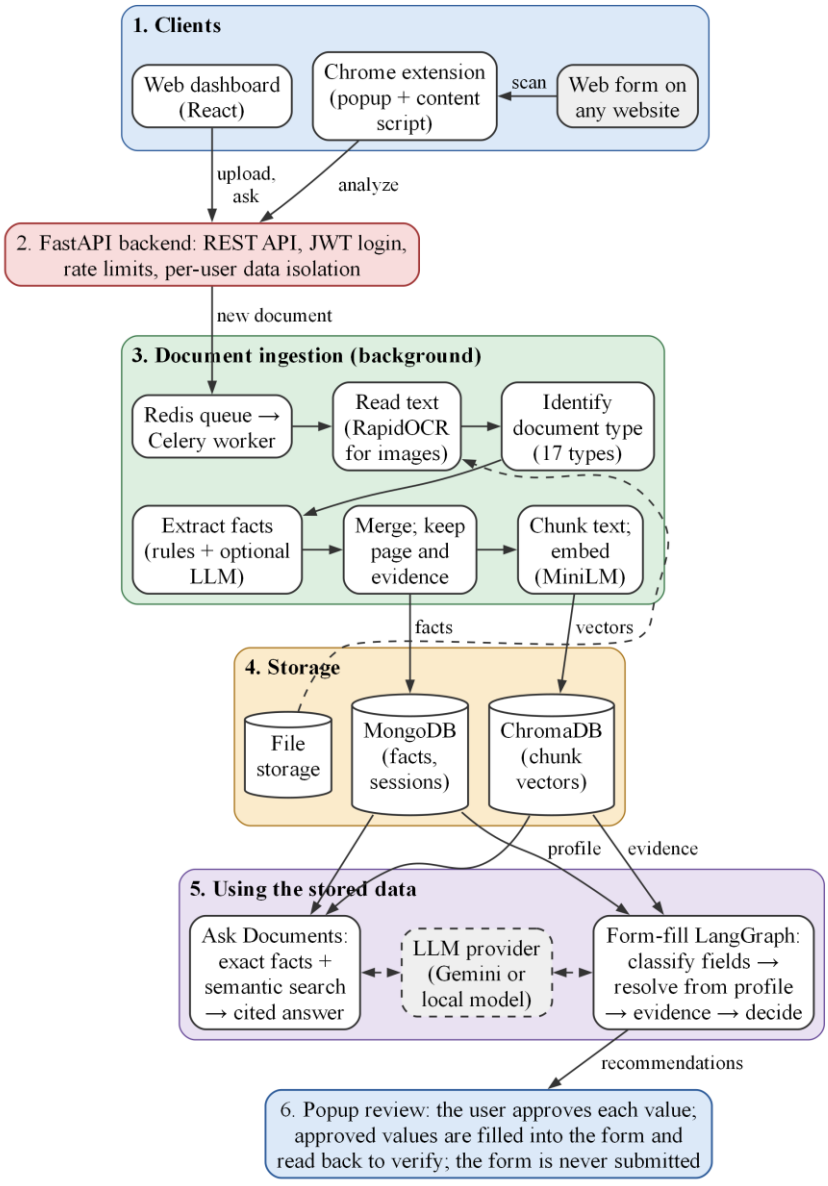


Figure 4.1: End-to-end architecture and data flow, from document upload to reviewed form filling

1. **Clients:** the user works through a web dashboard (upload, profile, questions) and a Chrome extension. The extension reads only the descriptions of a form's fields, never their current values.
2. **Backend:** a FastAPI server checks the user's token, applies rate limits and passes the user's identifier into every database query, so that one user can never read another user's data.

3. **Ingestion:** a new document is queued in Redis and processed by a Celery worker: its text is read (with OCR for images), its type is identified, facts are extracted and merged with their evidence, and the text is split into chunks and embedded.
4. **Storage:** the original file is kept in file storage, the facts and form sessions in MongoDB, and the chunk embeddings in ChromaDB.
5. **Using the stored data:** Ask Documents answers questions from exact facts first and from semantic search over the chunks second; the form-fill pipeline classifies form fields, takes values from the profile and, where needed, from cited document evidence. Both may call the LLM, but neither accepts an answer that is not supported by the stored data.
6. **Review:** the recommendations return to the extension popup, where the user approves, edits or skips each value before anything is filled.

4.2 DATA FLOW

Figure 4.2 is the level-1 data-flow diagram. It shows six processes, the data stores they read and write, and the optional LLM provider as an external entity.

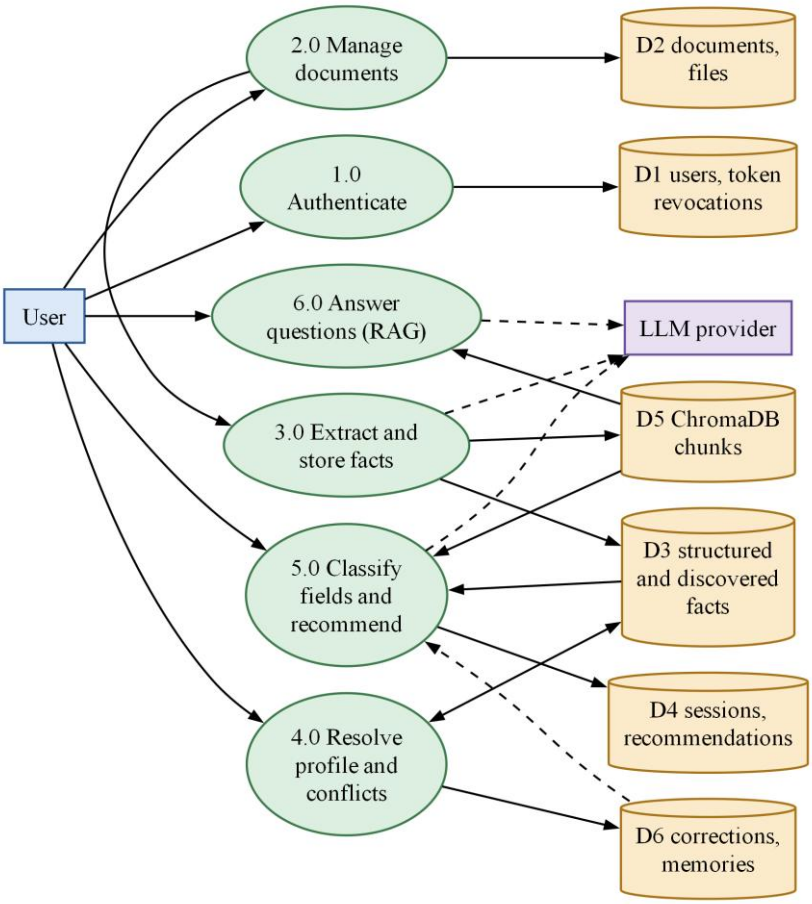


Figure 4.2: Data-flow diagram, level 1

The user signs in (1.0) and uploads documents (2.0). Each stored file is passed to fact extraction (3.0), which writes facts to the fact store D3 and chunk embeddings to D5. Profile and conflict resolution (4.0) lets the user confirm or correct facts and records each correction in D6. Field classification and recommendation (5.0) reads the facts, the document chunks and saved answers, and writes form sessions and recommendations to D4. Question answering (6.0) reads the chunks and facts. Only processes 3.0, 5.0 and 6.0 may call the LLM.

4.3 DATABASE DESIGN

As explained in Section 2.1.1, MongoDB stores records as flexible documents grouped into collections. Almost every record carries the owner's *user_id*, and every query and deletion includes it. A separate ChromaDB collection holds the chunk

embeddings, labelled with the same *user_id* and *document_id* so that search results always belong to the person asking.

Figure 4.3 shows the entity–relationship diagram. A user owns documents, structured facts, form sessions, corrections and saved answers; a document owns its processing records and facts; a form session owns one recommendation per field.

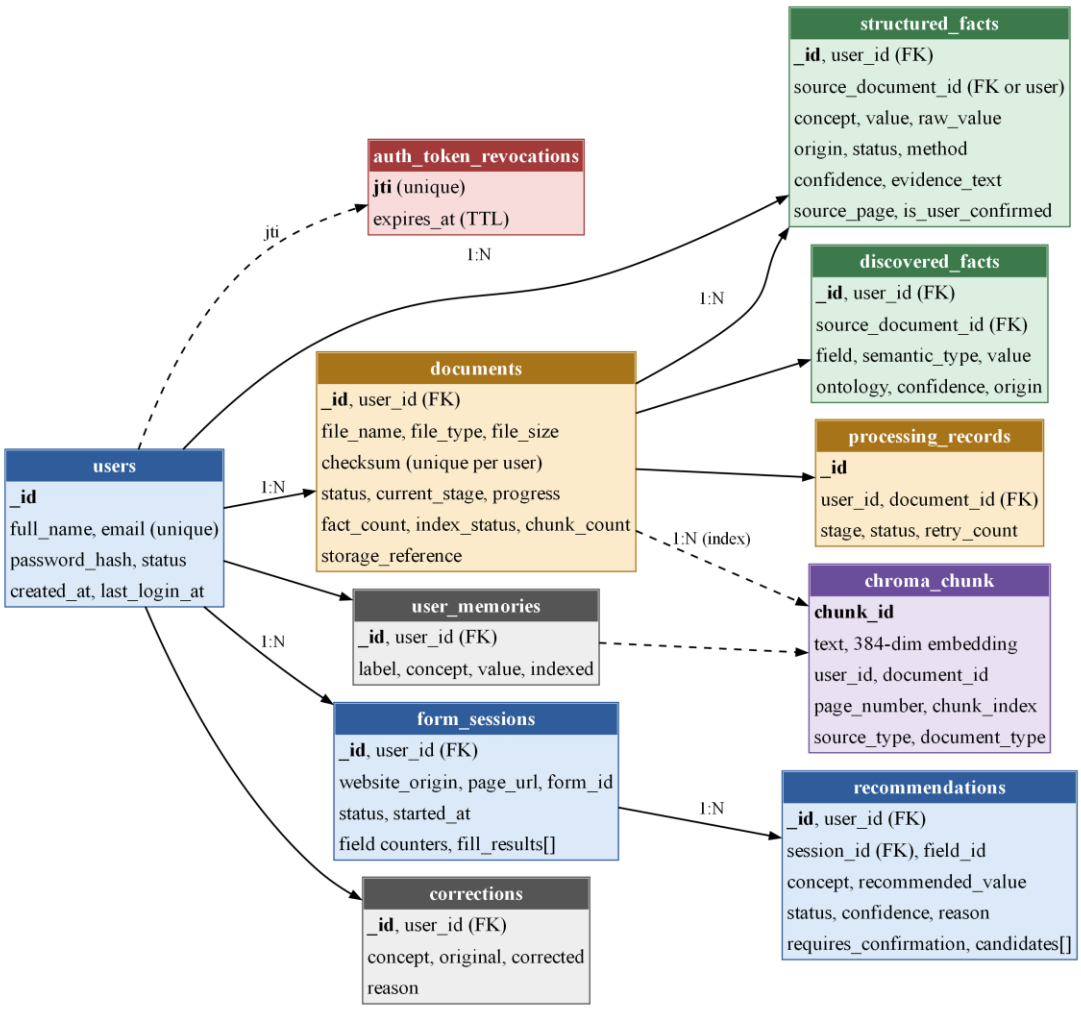


Figure 4.3: Entity–relationship diagram of the MongoDB collections and the ChromaDB chunk index

Two design decisions keep the data consistent. First, conflicts are not stored: they are computed when needed from the active facts, so deleting or reprocessing a document can never leave an out-of-date conflict behind. Second, reprocessing a document first deletes its old facts and chunks, so nothing is duplicated. Tables extracted from documents, such as subject-wise marks or bank transactions, are kept as *discovered facts*, while form-fillable concepts become *structured facts*.

4.4 FORM-FILLING PIPELINE

Form analysis is implemented as a LangGraph state graph [18] of seven steps that share one state object. Two conditional steps are skipped when they are not needed, which keeps most analyses free of LLM calls:

1. **sanitize:** blocks password, payment, OTP, CAPTCHA, hidden and file fields, and sets aside fields whose text looks like an injected instruction.
2. **deterministic_classify:** maps each field to a concept using its autocomplete attribute, label, placeholder, name and nearby text, matched against the ontology aliases. A label inside a “Father’s details” section is not given the user’s own name.
3. **llm_classify (only if needed):** sends one batched request for the fields the rules could not resolve; the answer is accepted only if it names a known concept with enough confidence.
4. **resolve_profile:** takes values from the user’s profile, marking a concept with disagreeing documents as needing review.
5. **resolve_documents (only if needed):** retrieves document chunks for fields still empty and asks the LLM for a value that must appear word for word in a cited chunk; anything else is discarded.
6. **decide and persist:** the decide step applies the rules of Section 4.5, and the persist step stores one recommendation per field.

4.5 DECISION RULES

Figure 4.4 is the flowchart of the decide step, which gives every field its final status. Each path ends at a single End terminator, where the recommendation is stored.

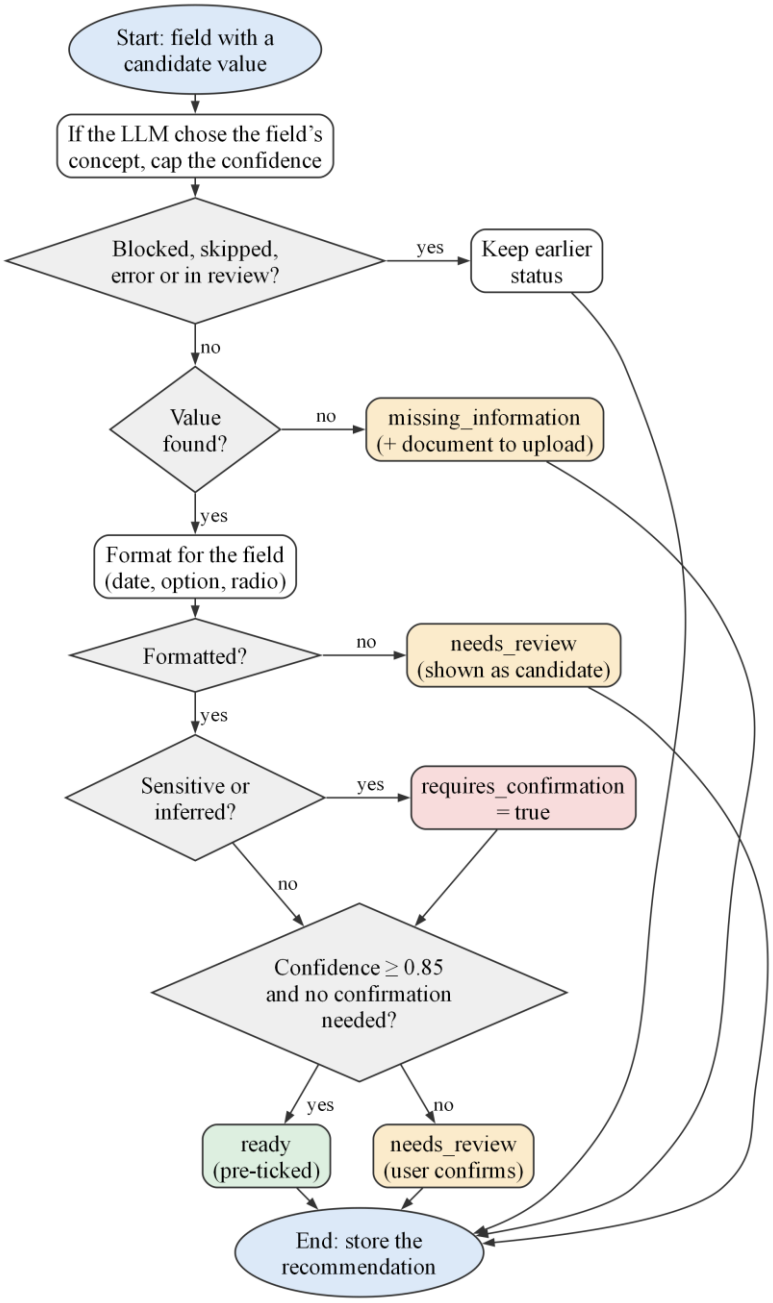


Figure 4.4: Flowchart of the decide step of the form-filling pipeline

A field that was already blocked keeps its status, and a field without a value becomes *missing information*, with a hint about which document usually contains it. Otherwise the value is formatted for the field; if this fails, the value is only shown as a candidate. Sensitive concepts and inferred values always require confirmation. A field is *ready*, and therefore pre-ticked in the popup, only when its confidence is at least 0.85 and no

confirmation is required; every other field is *needs review*. When the LLM chose the field's concept, the value's confidence is capped by the confidence of that choice, so an uncertain mapping cannot produce a ready value.

4.6 SECURITY DESIGN

- **Accounts:** passwords are hashed with bcrypt [32]; access tokens are JSON Web Tokens [33] valid for 30 minutes and revoked on sign-out; repeated failed logins are rate-limited.
- **Isolation:** every MongoDB and ChromaDB operation is filtered by the user's identifier.
- **Uploads:** the file extension, declared type, content signature (magic bytes), size limit of 10 MB and duplicate checksum are all checked.
- **Prompt injection:** page and document text is wrapped in clearly marked data blocks, six injection patterns are detected, and LLM output must cite stored evidence [13], [14].
- **Browser:** the extension uses on-demand page access, keeps its token away from page scripts, never reads field values and never submits.

CHAPTER SUMMARY

The design was presented through the end-to-end architecture, the level-1 data flow, the database design, the seven-step form-filling pipeline, the decision flowchart and the security controls. The main design decisions are a single ontology, rules before models, answers grounded in stored evidence, conflicts computed rather than stored, per-user isolation and mandatory user review.

Chapter 5
RESULT ANALYSIS: SNAPSHOTS

This chapter shows the working system and analyses its measured results. All screenshots were captured on 22 October 2026 from the running system with Playwright [36]. They use a dedicated demonstration account loaded with fifteen **synthetic** documents for a fictional applicant; no real personal data appears in this report. The measured results come from the evaluation run of 16 October 2026 on the same data, using the Gemini model and the all-MiniLM-L6-v2 embedding model.

5.1 DASHBOARD AND DOCUMENT PROCESSING

Figure 5.1 shows the dashboard after all fifteen documents were processed. It reports 15 documents, 90 profile values (13 confirmed by the user), 11 conflicts to resolve and 19 field reviews pending. Each card links to the page where the user can act on it.

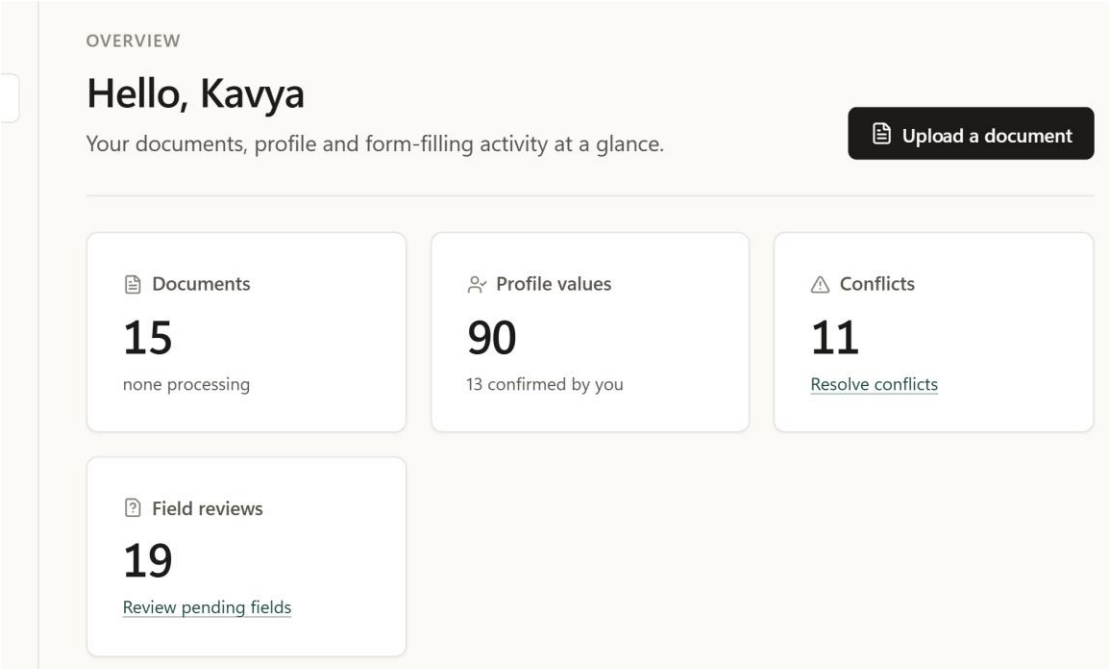


Figure 5.1: Dashboard overview of the demonstration account

Figure 5.2 shows the details of the Class 12 mark sheet. The system identified the document type, and for each value it lists how the value was obtained (Extracted or Normalised), the confidence, and the exact evidence text with its page. This evidence trail is the basis of Objective 1.

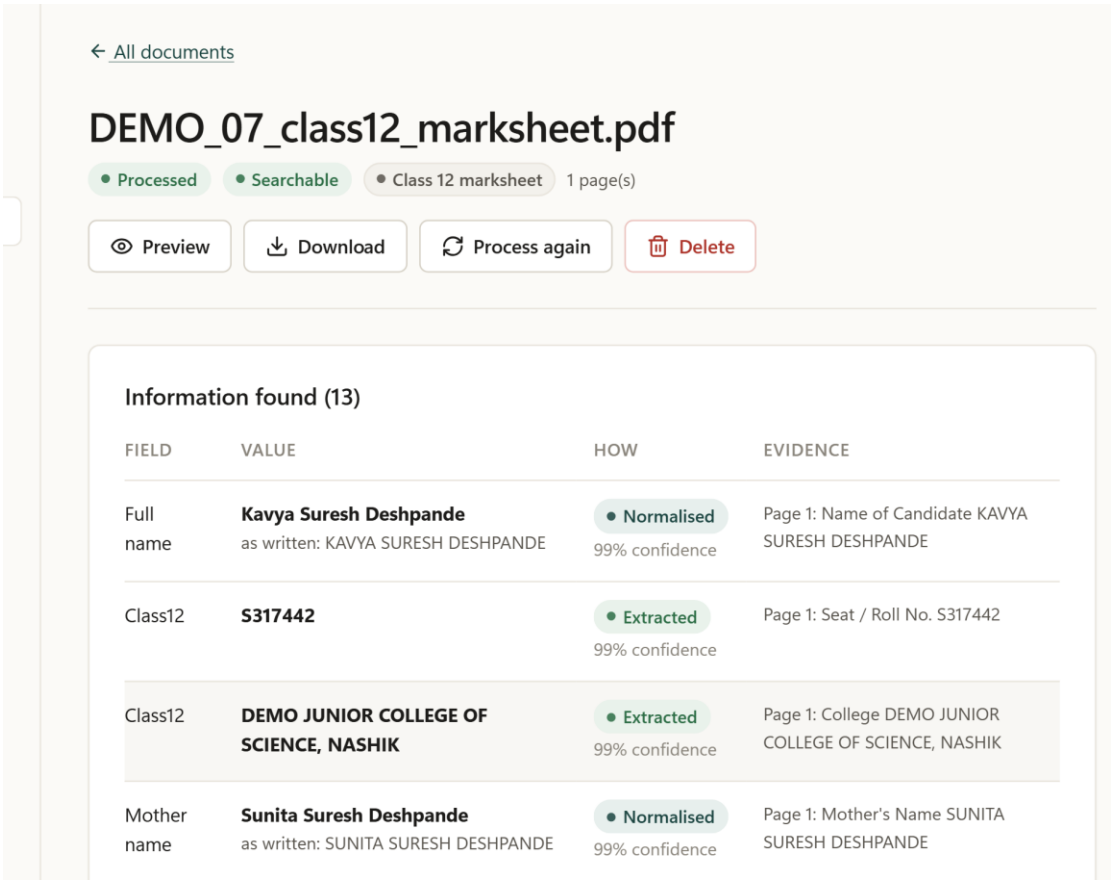


Figure 5.2: Facts extracted from the Class 12 mark sheet, with evidence

5.2 PROFILE AND CONFLICTS

Figure 5.3 shows the Profile page, which groups values by topic. Each value lists the documents that support it; the father's name, for example, is supported by eight documents. The user can confirm, edit or delete each value.

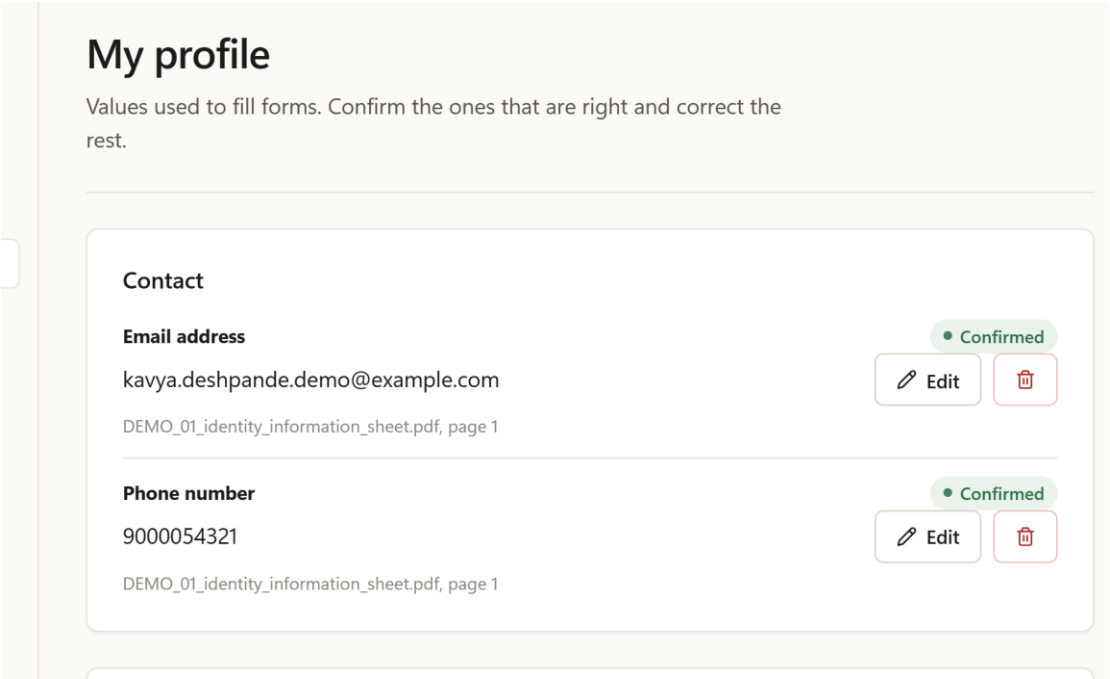


Figure 5.3: Profile page with the supporting documents of each value

Figure 5.4 shows a real disagreement found by the system: the caste validity certificate contains two different dates (20/11/2025 and "Date of Issue 09/07/2025"). Both candidates are shown with their evidence, and nothing is filled until the user chooses (Objective 2).

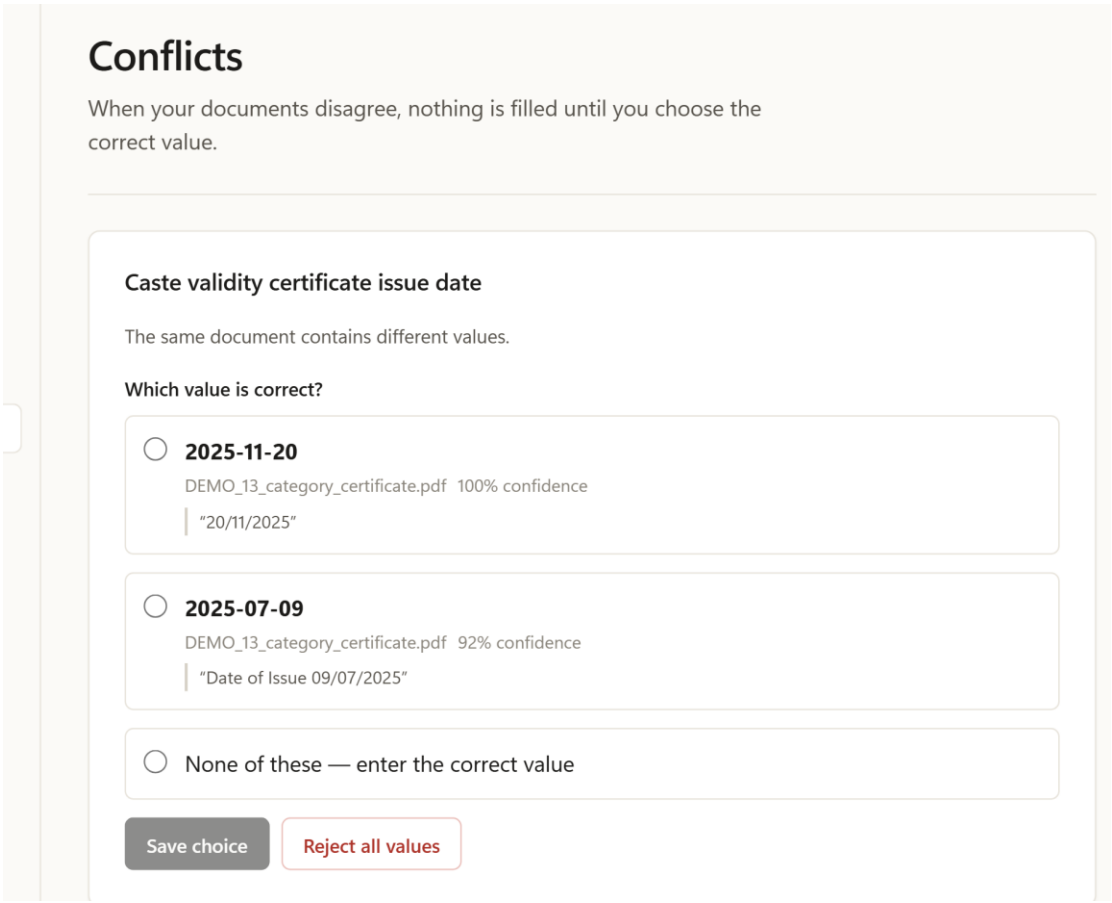


Figure 5.4: Conflict between two dates found in one certificate

5.3 ASK DOCUMENTS

Figure 5.5 shows the answer to "What was my Class 12 percentage, and which document states it?". The answer (87.20%) comes from an exact fact, and the Sources panel quotes the cited passage of the Class 12 mark sheet (Objective 4).

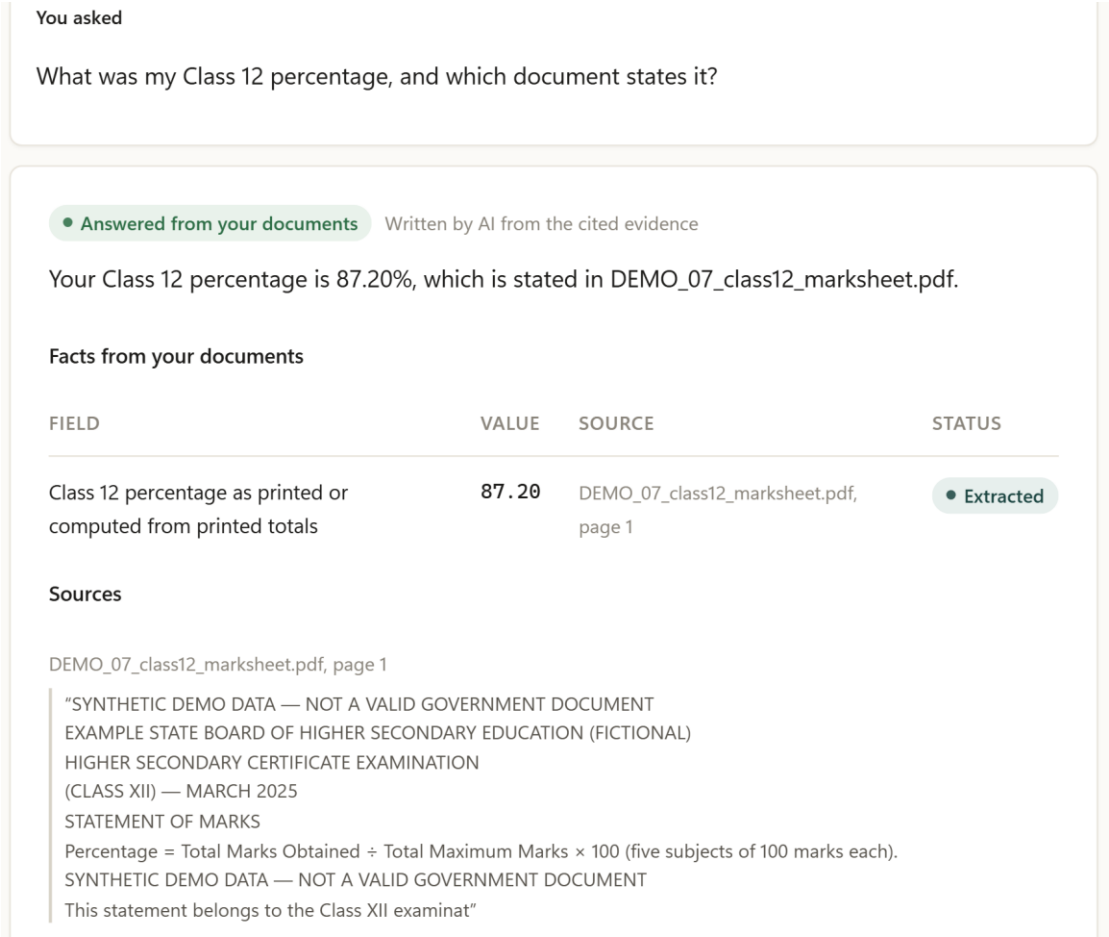


Figure 5.5: Ask Documents: answer with the cited source

5.4 BROWSER EXTENSION

Figure 5.6 shows the extension on a synthetic Class 12 application form. On the left, after analysis, 23 of 25 fields are ready and 2 need attention because the passing month and result are not in the documents. On the right, after the user chose Fill, all 23 values were filled and read back from the page, and the popup confirms that nothing was submitted.

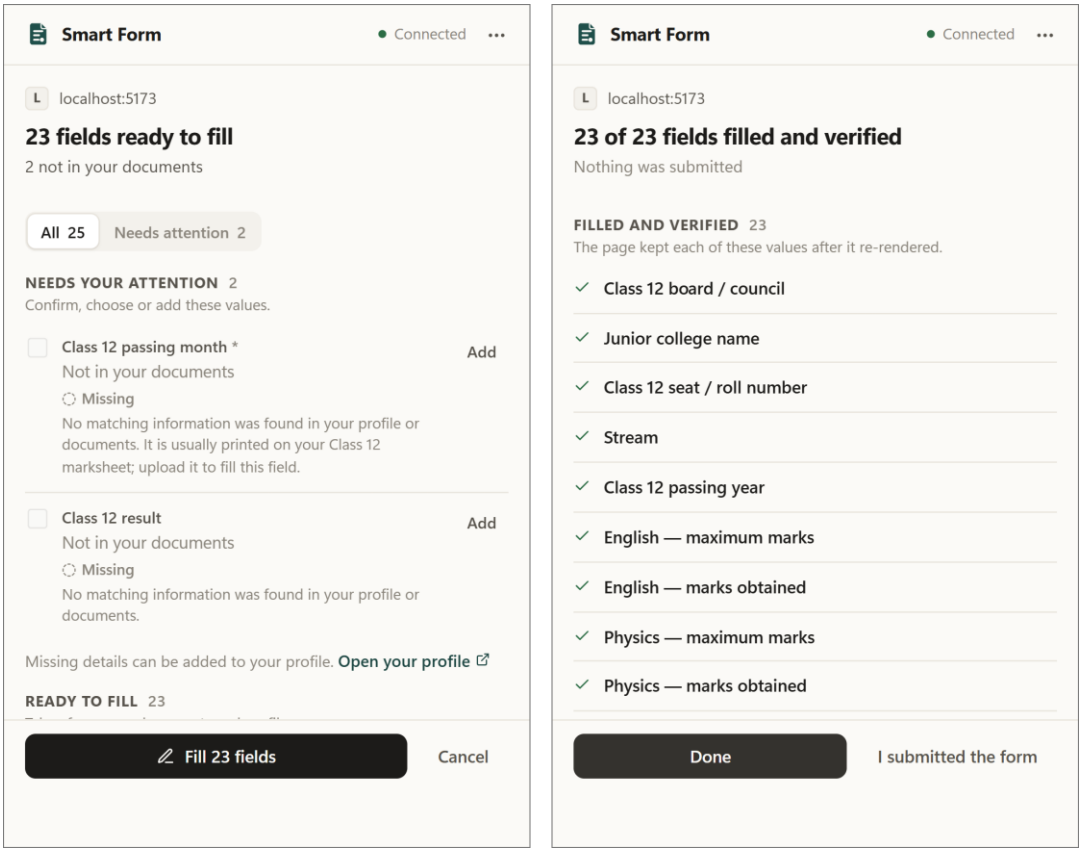


Figure 5.6: Extension popup: review before filling (left) and verified result (right)

Figure 5.7 shows part of the filled form. The subject-wise marks were filled and verified, but it also shows a mistake: the "Elective language" row, which the mark sheet does not contain, received the total marks (500 and 436). Errors of this kind are counted in Table 5.2.

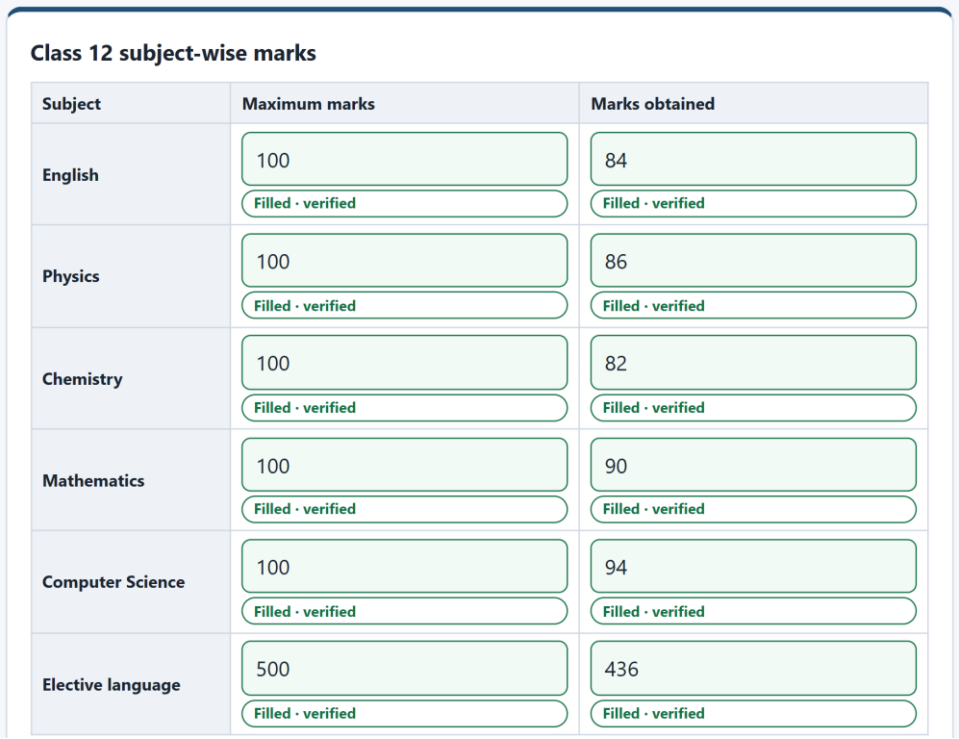


Figure 5.7: Filled subject-wise marks on the synthetic Class 12 form

5.5 MEASURED RESULTS

Table 5.1 summarises extraction, retrieval and question answering on the fifteen synthetic documents. The answers were compared with a hidden answer key that the system never reads.

Table 5.1: Extraction, retrieval and question-answering results (15 synthetic documents)

Measure	Result
Document type identified correctly	15 / 15 (100%)
Expected fields extracted with the exact value	128 / 132 (97.0%)
Fields from scanned images / from digital PDFs	97.8% / 96.6%
Table rows exactly right (subject marks, transactions, members)	40 / 40
Documents stored in MongoDB / indexed in ChromaDB	15 / 15 and 15 / 15
Retrieval: correct document ranked first / within the top four	93.3% / 100% (15 queries)
Ask Documents questions answered correctly	28 / 28
Answers citing the correct source document	100% (23 questions with an expected source)
Forbidden or invented values in answers; median answer time	0; 0.08 s

Table 5.2 gives the outcome of filling the fifteen matching forms through the browser extension.

Table 5.2: Browser autofill results on 15 synthetic forms

Outcome on 15 synthetic forms (real extension, real backend)	Count
Fields filled correctly	112
Fields filled incorrectly (including 3 filled when they should stay empty)	9
Expected values held for the user's review (not filled)	84
Expected values missed silently	0
Expected fields not detected on the page	2
Fields whose value is absent from the documents, correctly left empty	12 of 15
Fields reserved for the user's own choice that the system touched	0
Forms submitted by the system	0

Of the 121 values the system filled automatically, 112 were correct, a precision of 92.6%. The system preferred caution: 84 expected values were held for review rather than filled (for example sensitive values, values from disagreeing documents and values below the 0.85 confidence threshold), and no form was submitted. The OCR benchmark on 30 rendered documents gave field recall of 0.948 for clean scans, 0.850 for pages skewed by 3–4°, 0.948 for pages rotated by 90° and 0.855 for simulated phone photographs, at 2.6 to 7.8 seconds per page on a CPU. In a comparison of language models on six grounded questions and eight hard field-classification cases, Gemini answered 6/6 and 8/8, the local qwen2:7b model 4/6 and 5/8, and gemma:2b 2/6 and 1/8, which is why Gemini is the primary model and the local model is only a fallback whose mappings always need review.

5.6 ANALYSIS

- **Objectives 1 and 4:** extraction (97.0% exact fields) and question answering (28/28 with sources) met their goals on the synthetic set.
- **Objective 2:** disagreements, such as the two issue dates in Figure 5.4, were shown to the user instead of being resolved silently.
- **Objective 3:** 112 correct fills against 9 errors. Three errors mixed up the correspondence and permanent addresses (two PIN codes and one address), three gave the wrong relationship for household members, and three placed a value in a field that should have stayed empty, including the elective-language row of Figure 5.7.

- **Objective 5:** no form was submitted, no field reserved for the user was touched, and every filled value was read back from the page.
- **Limits of the evidence:** the documents are synthetic and the sets are small, so the percentages are indicative, not population accuracy.

CHAPTER SUMMARY

Seven snapshots showed the working system from processed documents to a filled and verified form, including one real disagreement and one real filling error. On fifteen synthetic documents and forms, the system identified every document type, extracted 97.0% of expected fields, answered all 28 questions with sources and filled 112 fields correctly and 9 incorrectly, without submitting any form.

Chapter 6
TESTING

This chapter describes how the system was tested: the strategy and environment, the test inventory, representative test cases, and the results. Every number is taken from an actual test run; nothing was estimated. Tests that were not run are stated as such.

6.1 TESTING STRATEGY

Testing is organised in four levels. **Unit and API tests** check one module or one server endpoint at a time. **Integration tests** run against a real ChromaDB service with the real embedding model. **Browser end-to-end tests** load the built extension into a real Chromium browser with Playwright [36] and fill a test form through the whole system. Finally, the **evaluation** of Chapter 5 measures accuracy on synthetic documents and forms.

The default backend test run never touches the developer's own data and never calls a paid service. The test configuration forces a *mock* language model (a stand-in that returns fixed, valid answers), a temporary file store and a randomly named database that is deleted afterwards. Tests that need ChromaDB or a real LLM provider are marked and run only on request. A continuous-integration workflow runs the unit, integration, dashboard, extension and browser tests on every change pushed to the repository.

6.2 TEST ENVIRONMENT

All tests were run on a laptop with an Intel Core i5-13450HX processor (10 cores) and 16 GB of memory (15.7 GB usable), running Windows 11 Home, Python 3.11.9, Node.js 22.14.0, Docker 29.4.0 with Compose v2, MongoDB 8 and ChromaDB in Docker containers, and Playwright-managed Chromium. Backend tests use pytest; dashboard and extension tests use Vitest with a simulated browser page (jsdom).

6.3 TEST INVENTORY

Table 6.1 groups the 301 backend tests by area, with the number of tests in each area, and lists the dashboard and extension suites. The counts were collected from the test runner itself.

Table 6.1: Automated test inventory

Area	What is tested	Tests
Accounts and configuration	Registration, login, token expiry and revocation, rate limits, unsafe settings rejected	21
Documents and processing	Valid and invalid uploads, duplicates, corrupted files, reprocessing, deletion, isolation	19
Indian documents and OCR	17 document types, identifiers, mark-sheet tables, OCR layout, chunking	93
Extraction and ontology	Rule-based extraction, normalisation, merging, alias matching	32
Profile and conflicts	Conflict detection, resolution, corrections, deleting all data	10
Form filling	Field classification, recommendations, formatting, decisions, exam forms	62
Ask Documents and RAG	Question planning, grounded answers, sources, cross-user protection	41
LLM and prompt safety	Injection patterns, untrusted blocks, provider errors and retries	16
Platform	Health checks, error format, cross-origin rules, extension manifest	7
Backend total		301
Dashboard (7 files)	Sign-in, documents, profile, Ask page, exam readiness, demo data, badges	30
Extension (8 files)	Field detection, filling, uploads, document types, API client, popup flow	112

6.4 TEST CASES

Table 6.2 lists representative test cases. The expected result is the assertion made by the named automated test, and the actual result is the outcome of the run of 22 October 2026.

Table 6.2: Representative test cases

ID	Test case (automated test)	Expected result	Actual result	Status
TC-01	Wrong password and unknown e-mail	Same error for both	Same 401 error	Pass
TC-02	Token sent in the query string	Request refused	Refused	Pass
TC-03	Upload a renamed executable or fake PDF	Rejected; nothing stored	Rejected	Pass
TC-04	Upload the same file twice	Duplicate reported	Duplicate reported	Pass
TC-05	Index fails during processing	Facts kept; index marked failed	As expected	Pass
TC-06	Two documents disagree on a value	Conflict listed with sources	Listed with sources	Pass
TC-07	“Name” inside a father’s section	User’s own name not used	Not used	Pass
TC-08	Field with no fact and no evidence	Missing; no value invented	Missing	Pass
TC-09	Field text says “ignore previous instructions”	Field quarantined; not sent to LLM	Quarantined	Pass
TC-10	LLM value not present in cited text	Value discarded	Discarded	Pass
TC-11	Class 12 question with both mark sheets	Never answered from Class 10	Correct mark sheet	Pass
TC-12	User B asks for user A’s data	Refused or not found	Refused	Pass
TC-13	Page with password, hidden and honeypot fields	Those fields not listed	Not listed	Pass
TC-14	Fill a framework-controlled input	Value kept and verified	Kept and verified	Pass
TC-15	Complete fill in real Chromium	Values filled; form not submitted	Not submitted	Pass

6.5 TEST RESULTS

Table 6.3 summarises the results. For each suite it shows the total number of test cases, how many passed and failed, and the pass percentage, as the reviewer requested. Seven backend tests are deselected by default: three need ChromaDB and are run separately (row 2), and four call a paid LLM provider and were not run.

Table 6.3: Test results: total, passed and failed test cases

Test suite (tool)	Run date	Total	Passed	Failed	Pass %
Backend unit and API (pytest, mock LLM)	22 Oct 2026	301	301	0	100%
Vector-database integration (pytest, real ChromaDB)	22 Oct 2026	3	3	0	100%
Dashboard (Vitest)	22 Oct 2026	30	30	0	100%
Extension (Vitest)	22 Oct 2026	112	112	0	100%
Browser end-to-end (Playwright, Chromium)	15 Oct 2026	7	7	0	100%
Real LLM provider tests (opt-in)	Not run	4	–	–	–
Total executed		**453**	**453**	**0**	**100%**

All 446 tests run on 22 October 2026 passed, and the seven browser tests passed in their recorded run of 15 October 2026; they were not repeated for this report. The evaluation of Chapter 5 is a separate measurement of accuracy and is not counted as test cases here.

6.6 DISCUSSION

The tests are strongest where the design makes promises: refusal paths, per-user isolation, grounding of LLM output and the rule that nothing is filled or submitted without the user. Penetration testing, load testing, accessibility testing and tests with the paid LLM provider were not carried out, and OCR was evaluated only on rendered synthetic images, not on real scans. These gaps are listed as future work in Chapter 7.

CHAPTER SUMMARY

The system was tested at four levels. On 22 October 2026, 301 backend, 3 vector-database, 30 dashboard and 112 extension tests all passed; the 7 browser end-to-end tests passed in the recorded run of 15 October 2026. In total 453 of 453 executed test cases passed and none failed. Real-provider, load and penetration tests were not run.

Chapter 7
CONCLUSION & FUTURE WORK

This chapter gives the single conclusion of the report: what was built, how far the five objectives of Chapter 1 were met, what was learned, which limitations remain, and what should be done next.

7.1 CONCLUSION

The project set out to build a privacy-first assistant that fills web forms from a person's own documents while keeping every value traceable and every decision with the user. The resulting Smart Form Auto-Filling System consists of a FastAPI backend with MongoDB and ChromaDB, a React dashboard and a Chrome extension, and starts with a single Docker Compose command. Documents in PDF, Word, text and image form are read, classified into seventeen Indian document types and converted into facts from a 220-concept ontology, each with its evidence. A seven-step LangGraph pipeline matches form fields to these facts, and the extension fills only the values the user approves.

7.1.1 Achievement of Objectives

Table 7.1 compares each objective with the evidence from Chapters 5 and 6.

Table 7.1: Achievement of the objectives

Objective	Status	Evidence
1. Document understanding with evidence	Achieved	Type identified for 15/15 documents; 97.0% of fields exact (Table 5.1)
2. Verified profile with conflicts	Achieved	Conflicts shown with sources (Figure 5.4); test TC-06
3. Semantic form filling	Achieved, with errors	112 correct and 9 incorrect fills; 84 held for review (Table 5.2)
4. Grounded question answering	Achieved	28/28 questions correct with sources (Table 5.1)
5. Safety and verification	Achieved	No form submitted; 453/453 tests passed (Table 6.3)

7.1.2 Principal Findings

- **Rules before models:** deterministic matching over a shared ontology resolves most fields, so the language model is needed only for the unclear remainder, which keeps cost, delay and risk low.
- **Disagreements must be visible:** real document sets disagree; showing the conflict was more reliable than any automatic choice.
- **Grounding checks are cheap and effective:** requiring a value to appear in the cited text removes a class of invented answers without a second model call.
- **Caution has a price:** holding 84 values for review avoided wrong fills but asks more of the user; the remaining errors show where field matching needs more context.

7.2 LIMITATIONS AND FUTURE WORK

Table 7.2 pairs each known limitation with the work proposed to address it.

Table 7.2: Limitations and proposed future work

Current limitation	Proposed future work
Accuracy measured only on synthetic documents and forms	Build an anonymised benchmark of real documents and forms and report accuracy per concept
Errors with similar fields (correspondence and permanent address, household relationships)	Use section headings and the form's structure more strongly, and ask the user when two concepts compete
English OCR only; Indian scripts and handwriting are not read	Add multilingual OCR models and evaluate them on real scans
Chrome only; fields added after the scan need a new scan	Port to other Manifest V3 browsers; watch the page for new fields and multi-step forms
Examination requirement profiles not verified against official bulletins	Verify each profile against the current official notification before use
No load, penetration or accessibility testing	Add these tests and run the paid-provider tests with synthetic data

Three further extensions are promising. First, a fully local language model would keep all document text on the user's computer (an optional local fallback exists but is less accurate). Second, graph-based retrieval such as HippoRAG [27] could answer questions that combine several facts. Third, a per-site consent list would let the user trust selected portals while keeping the review step for every fill.

CHAPTER SUMMARY

All five objectives were met and all 453 executed tests passed. The main limitations, synthetic-only evaluation, confusion between similar fields and English-only OCR, define the future work.

References

[1] X. Lin, P. Ilia, and J. Polakis, "Fill in the blanks: Empirical analysis of the privacy threats of browser form autofill," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, Virtual Event, USA: ACM, Nov. 2020, pp. 507–519, doi: 10.1145/3372297.3417271.

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, NeurIPS Foundation, 2020, pp. 9459–9474.

[3] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, "WebArena: A realistic web environment for building autonomous agents," in *Proc. 12th Int. Conf. Learning Representations (ICLR)*, Vienna, Austria, 2024. [Online]. Available: arXiv:2307.13854

[4] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, "Mind2Web: Towards a generalist agent for the web," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, New Orleans, LA, USA: NeurIPS Foundation, 2023, pp. 28091–28114, doi: 10.52202/075280-1220.

[5] H. He, W. Yao, K. Ma, W. Yu, Y. Dai, H. Zhang, Z. Lan, and D. Yu, "WebVoyager: Building an end-to-end web agent with large multimodal models," in *Proc. 62nd Annu. Meeting Assoc. Computational Linguistics (ACL), Vol. 1: Long Papers*, Bangkok, Thailand: ACL, 2024, pp. 6864–6890, doi: 10.18653/v1/2024.acl-long.371.

[6] A. Abdallah, D. Eberharter, Z. Pfister, and A. Jatowt, "Transformers and language models in form understanding: A comprehensive review of scanned document analysis," arXiv preprint arXiv:2403.04080, 2024.

[7] Y. Du, C. Li, R. Guo, X. Yin, W. Liu, J. Zhou, Y. Bai, Z. Yu, Y. Yang, Q. Dang, and H. Wang, "PP-OCR: A practical ultra lightweight OCR system," arXiv preprint arXiv:2009.09941, 2020.

[8] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. 2019 Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China: ACL, 2019, pp. 3982–3992, doi: 10.18653/v1/D19-1410.

[9] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, NeurIPS Foundation, 2020. [Online]. Available: arXiv:2002.10957

[10] B. Zheng, B. Gou, J. Kil, H. Sun, and Y. Su, "GPT-4V(ision) is a generalist web agent, if grounded," in *Proc. 41st Int. Conf. Machine Learning (ICML)*, PMLR, vol. 235, Vienna, Austria, 2024, pp. 61349–61385.

[11] D. Nguyen *et al.*, "GUI agents: A survey," in *Findings of the Assoc. Computational Linguistics: ACL 2025*, Vienna, Austria: ACL, 2025, pp. 22522–22538, doi: 10.18653/v1/2025.findings-acl.1158.

[12] L. Wang *et al.*, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, Art. no. 186345, 2024, doi: 10.1007/s11704-024-40231-1.

[13] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, Copenhagen, Denmark: ACM, Nov. 2023, pp. 79–90, doi: 10.1145/3605764.3623985.

[14] OWASP Foundation, "OWASP Top 10 for LLM Applications 2025." Accessed: Oct. 22, 2026. [Online]. Available: https://genai.owasp.org/llm-top-10/

[15] MongoDB, Inc., "What is MongoDB? – Database Manual (version 9.0)." Accessed: Oct. 22, 2026. [Online]. Available: https://www.mongodb.com/docs/manual/

[16] Chroma, "Chroma documentation: Introduction." Accessed: Oct. 22, 2026. [Online]. Available: https://docs.trychroma.com/

[17] S. Ramírez, "FastAPI documentation." Accessed: Oct. 22, 2026. [Online]. Available: https://fastapi.tiangolo.com/

[18] LangChain, Inc., "LangGraph overview." Accessed: Oct. 22, 2026. [Online]. Available: https://docs.langchain.com/oss/python/langgraph/overview

[19] Google LLC, "Extensions / Manifest V3," Chrome for Developers. Accessed: Oct. 22, 2026. [Online]. Available: https://developer.chrome.com/docs/extensions/develop/migrate/what-is-mv3

[20] W. Hong, W. Wang, Q. Lv, J. Xu, W. Yu, J. Ji, Y. Wang, Z. Wang, Y. Dong, M. Ding, and J. Tang, "CogAgent: A visual language model for GUI agents," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, Seattle, WA, USA: IEEE, 2024, pp. 14281–14290, doi: 10.1109/CVPR52733.2024.01354.

[21] K. Cheng, Q. Sun, Y. Chu, F. Xu, Y. Li, J. Zhang, and Z. Wu, "SeeClick: Harnessing GUI grounding for advanced visual GUI agents," in *Proc. 62nd*

Annu. Meeting Assoc. Computational Linguistics (ACL), Vol. 1: Long Papers, Bangkok, Thailand: ACL, 2024, pp. 9313–9332, doi: 10.18653/v1/2024.acl-long.505.

[22] K. You, H. Zhang, E. Schoop, F. Weers, A. Swearngin, J. Nichols, Y. Yang, and Z. Gan, "Ferret-UI: Grounded mobile UI understanding with multimodal LLMs," in *Computer Vision – ECCV 2024*, Lecture Notes in Computer Science, Cham, Switzerland: Springer, 2024, pp. 240–255, doi: 10.1007/978-3-031-73039-9_14.

[23] A. Drouin *et al.*, "WorkArena: How capable are web agents at solving common knowledge work tasks?," in *Proc. 41st Int. Conf. Machine Learning (ICML)*, PMLR, vol. 235, Vienna, Austria, 2024, pp. 11642–11662.

[24] D. Lee, J. Lee, K. Kim, J. Tack, J. Shin, Y. W. Teh, and K. Lee, "Learning to contextualize web pages for enhanced decision making by LLM agents," in *Proc. 13th Int. Conf. Learning Representations (ICLR)*, Singapore, 2025. [Online]. Available: arXiv:2503.10689

[25] Z. Zhang, X. Bo, C. Ma, R. Li, X. Chen, Q. Dai, J. Zhu, Z. Dong, and J.-R. Wen, "A survey on the memory mechanism of large language model-based agents," *ACM Trans. Information Systems*, vol. 43, no. 6, pp. 1–47, 2025, doi: 10.1145/3748302.

[26] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying large language models and knowledge graphs: A roadmap," *IEEE Trans. Knowledge and Data Engineering*, vol. 36, no. 7, pp. 3580–3599, Jul. 2024, doi: 10.1109/TKDE.2024.3352100.

[27] B. Jiménez Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, "HippoRAG: Neurobiologically inspired long-term memory for large language models," in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, Vancouver, BC, Canada: NeurIPS Foundation, 2024, pp. 59532–59569, doi: 10.52202/079017-1902.

[28] Y. Shao *et al.*, "FormLM: Recommending creation ideas for online forms by modelling semantic and structural information," in *Proc. 2022 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Abu Dhabi, UAE: ACL, 2022, pp. 8133–8149, doi: 10.18653/v1/2022.emnlp-main.557.

[29] H. Belgacem, X. Li, D. Bianculli, and L. Briand, "A machine learning approach for automated filling of categorical fields in data entry forms," *ACM Trans. Software Engineering and Methodology*, vol. 32, no. 2, pp. 1–40, 2023, doi: 10.1145/3533021.

[30] P. Alian, N. Nashid, M. Shahbandeh, and A. Mesbah, "Semantic constraint inference for web form test generation," in *Proc. 33rd ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA)*, Vienna, Austria: ACM, Sep. 2024, pp. 932–944, doi: 10.1145/3650212.3680332.

[31] RapidAI, “RapidOCR: OCR toolkit based on ONNX Runtime,” GitHub repository. Accessed: Oct. 22, 2026. [Online]. Available: https://github.com/RapidAI/RapidOCR

[32] N. Provos and D. Mazières, “A future-adaptable password scheme,” in *Proc. USENIX Annual Technical Conf.*, Monterey, CA, USA: USENIX Association, Jun. 1999, pp. 81–92.

[33] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” Internet Engineering Task Force, RFC 7519, May 2015, doi: 10.17487/RFC7519.

[34] Meta Platforms, Inc., “React documentation.” Accessed: Oct. 22, 2026. [Online]. Available: https://react.dev/

[35] Docker, Inc., “Docker Compose documentation.” Accessed: Oct. 22, 2026. [Online]. Available: https://docs.docker.com/compose/

[36] Microsoft Corp., “Playwright documentation: Installation.” Accessed: Oct. 22, 2026. [Online]. Available: https://playwright.dev/docs/intro

Appendix I: Source Code Excerpts

The excerpts below are copied unchanged from the project repository at the paths shown. Each implements a design decision described in Chapter 4, and a short explanation precedes it.

I.1 Construction of the form-filling graph

The seven steps of Section 4.4 are added as nodes. Two conditional edges skip LLM classification when every field is already resolved, and skip document retrieval when no field needs it. Source: *backend/app/agents/graph.py*.

```
graph = StateGraph(FillState)
graph.add_node("sanitize", self._sanitize)
graph.add_node("deterministic_classify", self._deterministic_classify)
graph.add_node("llm_classify", self._llm_classify)
graph.add_node("resolve_profile", self._resolve_profile)
graph.add_node("resolve_documents", self._resolve_documents)
graph.add_node("decide", self._decide)
graph.add_node("persist", self._persist)
graph.add_edge(START, "sanitize")
graph.add_edge("sanitize", "deterministic_classify")
graph.add_conditional_edges(
    "deterministic_classify",
    self._route_classification,
    {"llm": "llm_classify", "skip": "resolve_profile"},
)
graph.add_edge("llm_classify", "resolve_profile")
graph.add_conditional_edges(
    "resolve_profile",
    self._route_documents,
    {"rag": "resolve_documents", "skip": "decide"},
)
graph.add_edge("resolve_documents", "decide")
graph.add_edge("decide", "persist")
graph.add_edge("persist", END)
```

I.2 Prompt-injection patterns

Six regular expressions detect text that tries to give instructions to the model. A matching form field is set aside and a matching document chunk is dropped before any LLM call. Source: *backend/app/security/prompt_safety.py*.

```
_INJECTION_PATTERNS: tuple[re.Pattern[str], ...] = (
    re.compile(

r"\b(ignore|disregard|forget|override)\s+(all\s+|any\s+|the\s+)?(previous|pr
ior|above|earlier|system)\s+(instructions?|prompts?|rules?)",
        re.I,
    ),
    re.compile(
        r"\b(reveal|print|show|dump|output)\s+(all\s+|the\s+|any\s+)?"

r"((stored|user'?s?|private|personal|system|saved|other)\s+){1,2}(data|facts
|information|prompt|details|profile)",
        re.I,
    ),
    re.compile(r"\byou\s+are\s+now\s+(a|an|the)\b", re.I),
    re.compile(r"\bnew\s+(system\s+)?instructions?\s*:", re.I),
    re.compile(r"\b(system|developer)\s+prompt\b", re.I),
    re.compile(r"</?\s*(system|instruction|instructions|prompt|untrusted_[a-
z_]+)\s*>", re.I),
```

I.3 Writing a value into a web page

The extension sets the value through the browser's native setter and then fires input and change events, so that pages built with frameworks such as React register the new value. Source: *extension/src/content/autofill.ts*.

```
function setNativeValue(element: FieldElement, value: string): void {
  const prototype = Object.getPrototypeOf(element);
  const setter = Object.getOwnPropertyDescriptor(prototype, 'value')?.set;
  if (setter) setter.call(element, value);
  else element.value = value;
}

function announce(element: FieldElement): void {
  element.dispatchEvent(new Event('input', { bubbles: true }));
  element.dispatchEvent(new Event('change', { bubbles: true }));
}

function fillText(element: FieldElement, value: ApprovedValue): Attempt {
  element.focus({ preventScroll: true });
  setNativeValue(element, value.value);
  element.dispatchEvent(
    new InputEvent('input', { bubbles: true, inputType:
'insertReplacementText', data: value.value }),
  );
  element.dispatchEvent(new Event('change', { bubbles: true }));
  element.blur();
  return {
    result: { field_id: value.field_id, applied: true, verified: false },
    verify: () => (element.value === value.value
      ? { verified: true }
      : { verified: false, reason: 'The page changed or rejected the value'
}),
  };
}
```

Appendix II: List of Components

The software components used by the system are listed below; versions are those pinned or installed in the repository on 22 October 2026.

Table II.1 lists the components.

Table II.1: Software components and versions

Component	Version (installed or pinned)	Role in the system
Python / FastAPI / Pydantic	3.11.9 / 0.141.1 / 2.13.5	Backend server and request validation
MongoDB / Motor driver	8.0 (Docker image) / 3.7.1	Facts, documents and form sessions
ChromaDB	1.5.9	Vector index of document chunks
sentence-transformers	5.7.0 with all-MiniLM-L6-v2 (384 dimensions)	Text embeddings
RapidOCR on ONNX Runtime	rapidocr_onnxruntime 1.4.4; PP-OCRv3 detection, PP-OCRv4 recognition	OCR for scanned images
LangGraph	1.2.11	Form-filling state graph
Celery / Redis	5.6.3 / 7 (Alpine image)	Background document processing
Language model	Google gemini-3.5-flash-lite; optional qwen2:7b through Ollama	Unclear fields and answer wording
Dashboard	React 19.2, React Router 7, TypeScript 6.0, Vite 8	Web user interface
Extension	React 18.3, TypeScript 5.5, Vite 5.4, Manifest V3	Browser integration
Docker / Compose	29.4.0 / v2	Deployment of all services
pytest / Vitest / Playwright	8.4.2 / 5.0 (dashboard), 2.0 (extension) / 1.63	Automated testing

II.1 Installing and running the system

The steps below are taken from the project README. They start every service on one computer and load the extension into Chrome.

- **Prerequisites:** Docker Desktop with Compose v2, Python 3.11 (for tests outside Docker), Node.js 22 with npm, and Google Chrome. An LLM API key is optional; with *LLM_PROVIDER=mock* the system runs without any network call.

- **Configuration:** copy *.env.example* to *.env* and set *JWT_SECRET_KEY* to a random value of at least 32 characters; the backend refuses to start with the placeholder value. API keys must never be committed to the repository.
- **Optional OCR:** scanned images need the OCR engine, enabled with *INSTALL_OCR=true* and a rebuild of the images.

The commands start the services, check their health and build the extension:

```
docker compose up -d              # start all services
docker compose ps                 # wait until each shows "healthy"
npm ci                            # install dashboard and extension
cd extension && npm run build   # build into extension/dist
```

The dashboard then opens at *http://localhost:5173* and the API at *http://localhost:8000/api/v1*, with interactive documentation at *http://localhost:8000/docs*. To load the extension, open *chrome://extensions*, switch on Developer mode, choose **Load unpacked** and select the *extension/dist* folder.

Table II.2 lists the commands that run each automated test suite of Chapter 6.

Table II.2: Commands that run the automated test suites

Test suite	Command	What it checks
Backend (pytest)	cd backend && python -m pytest	Unit, API and pipeline tests with a mock LLM
Dashboard (Vitest)	cd frontend && npm test	Pages and components with mocked requests
Extension (Vitest)	cd extension && npm test	Field detection, filling, sensitivity rules
Browser end-to-end (Playwright)	cd extension && npm run test:e2e	Built extension in Chromium against a test backend

II.2 Main API endpoints

The backend exposes more than seventy routes; the complete list is generated by FastAPI at */docs*. Table II.3 lists the endpoints used in the workflow of Figure 3.1.

Table II.3: Main API endpoints used in the workflow

Method	Path	Purpose
POST	/api/v1/documents	Upload a document and queue it for processing
GET	/api/v1/documents/{id}/facts	Facts extracted from one document, with evidence
POST	/api/v1/retrieval/rag/answer	Answer a question from the user's documents (Ask Documents)
POST	/api/v1/forms/sessions/{id}/analyze	Run the form-filling pipeline on the detected fields
POST	/api/v1/forms/sessions/{id}/verify	Record which values were filled and read back
GET	/api/v1/health/services	Health of MongoDB, ChromaDB and Redis

Appendix III: List of Papers Presented and Published

[To be completed by the project team before printing: the title, authors, conference or journal, date and status (submitted, accepted, presented or published) of any paper based on this project. If no paper has been presented or published, write “None”.]